CML3112 WEEK 7

Preparing for Practical Test 1

Civil Engineering and Architecture

Practical revision before Test 1
Test time: confirm with the lecturer



WHAT TO DO

Your revision files

01 Use this slide deck and CML3112_Test1_practice.csv. Save both on your laptop.

02 Work through the examples in order in a new Colab notebook. Try each practice question before reading its answer.

03 Practice is ungraded. Keep your notebook for revision. No form submission is needed.

WHAT TO DO

Practice and test-day files

01 BEFORE THE TEST: use CML3112_Test1_practice.csv with the worked examples in this section.

02 ON TEST DAY: the entry page provides a different notebook and engineering_data.csv.

03 The test downloads unlock only during the scheduled test window.

WHAT TO DO

A new practice notebook in Colab

01 Open colab.research.google.com and sign in. Choose New notebook. Rename it Test revision.

02 Click Connect. Add a code cell with + Code. Run a cell with its triangle button.

03 Run cells from top to bottom. Save with File, then Save.

WHAT TO DO

Uploading your practice CSV

01 In Colab, click the folder icon on the left. Click Upload and choose CML3112_Test1_practice.csv.

02 Wait until the filename appears in the Files panel. Keep its name unchanged.

03 If the session resets, upload the file again before running your cells.

WHAT TO DO

What each column means

01 reading identifies one measurement. machine identifies the equipment that produced it.

02 load is the equipment load in percent. target is power demand, measured in kW.

03 ambient is the surrounding air temperature in °C. A blank cell means the value is missing.

WHAT TO DO

Reading a CSV with pandas

01 import pandas as pd loads pandas and gives it the short name pd.

02 pd.read_csv(filename) reads the uploaded table. The filename is text, so it needs quotation marks.

03 A name stores the resulting DataFrame. print(table) displays it, and len(table) counts its rows.

WORKED EXAMPLE

Loading the supplied practice file

```
import pandas as pd
filename = "CML3112_Test1_practice.csv"
source = pd.read_csv(filename)
print(source)
print(len(source))
```

Expected: seven rows.

HOW TO READ IT

source keeps the original file contents.

Blank numeric cells appear as NaN.

WHAT TO DO

The practice cleaning rules

01 Keep the first row for each reading identifier. Remove rows whose target is missing.

02 Keep load values from 0 to 100, including both limits.

03 After those removals, replace missing ambient values with the retained ambient median.

WHAT TO DO

Removing duplicates and missing targets

01 drop_duplicates("reading") keeps the first occurrence of each reading identifier.

02 dropna(subset=["target"]) removes a row when its target is missing.

03 .copy() creates a separate table. We keep source unchanged for comparison.

WORKED EXAMPLE

The first two cleaning steps

```
raw = source.drop_duplicates("reading")
print(len(raw))
raw = raw.dropna(subset=["target"]).copy()
print(len(raw))
```

HOW TO READ IT

Remove one repeated reading.

Remove one missing target.

Expected row counts: 6, then 5.

WHAT TO DO

Selecting a valid load range

01 between(0, 100) returns True for loads from 0 to 100. Other values receive False.

02 .loc[mask] keeps rows marked True. A mask is a row-by-row list of yes/no decisions.

03 The practice row with load 120 must be removed.

WORKED EXAMPLE

Keeping the valid loads

```
mask = raw["load"].between(0, 100)
raw = raw.loc[mask].copy()
print(raw["target"].tolist())
```

HOW TO READ IT

["load"] selects one column.

tolist displays its values as a list.

Expected retained targets: [10.0, 20.0, 30.0, 40.0].

WHAT TO DO

The median and missing ambient values

01 The median is the middle value after sorting the known values. Here: 20, 30, 40. Median: 30 °C.

02 .median() ignores missing values. .fillna(value) replaces only missing entries.

03 Calculate this median after removing the unwanted rows.

WORKED EXAMPLE

Filling the missing ambient value

```
median = raw["ambient"].median()
raw["ambient"] = raw["ambient"].fillna(median)
print(median)
print(raw["ambient"].tolist())
```

The median uses retained rows only.

Expected: 30.0 and [20.0, 30.0, 30.0, 40.0].

WHAT TO DO

Selecting, averaging and grouping

01 raw["target"] selects one column, called a Series.

02 .mean() adds the known values and divides by their count.

03 .groupby("machine") collects rows with the same machine label before calculating a result for each machine.

WORKED EXAMPLE

Overall mean and machine means

```
overall = raw["target"].mean()
groups = raw.groupby("machine")["target"].mean()
print(overall)
print(groups)
```

Select target, then calculate.

Expected: overall 25.0; M1 15.0; M2 35.0. All are in kW.

WHAT TO DO

Practice: calculate a changed mean

01 Using the four retained targets, imagine the last value changes from 40 to 60.

02 Calculate the overall mean and both machine means on paper.

03 Keep the CSV and notebook unchanged for the next example.

Practice answer

01 Overall: (10 + 20 + 30 + 60) / 4 = 30 kW.

02 M1: 15 kW. M2: (30 + 60) / 2 = 45 kW.

03 The four retained rows still end with target 40.

WHAT TO DO

Charts and numerical findings

01 A scatter plot shows pairs of values, such as load and target.

02 Label each axis with its quantity and unit. Give the chart a descriptive title.

03 For a finding, state the value, unit and what it describes. A pattern alone does not prove a cause.

WORKED EXAMPLE

A labelled practice chart

```
import matplotlib.pyplot as plt
plt.scatter(raw["load"], raw["target"])
plt.xlabel("Load (%)")
plt.ylabel("Target (kW)")
plt.title("Four practice readings")
plt.show()
```

HOW TO READ IT

scatter places one point per row.

xlabel and ylabel name the axes.

show displays the chart.

Expected points: (20, 10), (40, 20), (60, 30), (80, 40).

WHAT TO DO

Histograms and group-mean bars

01 A histogram counts readings within numeric ranges. plt.hist(values, bins=...) defines those ranges.

02 A bar chart compares named groups. plt.bar(labels, heights) places one bar per label.

03 The histogram shows the target distribution. The bars show one mean per machine.

WORKED EXAMPLE

A target histogram

```
plt.hist(raw["target"], bins=[0, 25, 50])
plt.xlabel("Target (kW)")
plt.ylabel("Number of readings")
plt.title("Practice target distribution")
plt.show()
```

HOW TO READ IT

bins gives the interval boundaries.

The y-axis counts records.

Expected: two readings in each of the two bins.

WHAT TO DO

Bar labels and heights

01 groups contains the mean for each machine. Its index holds the machine labels.

02 groups.index gives M1 and M2. groups.values gives 15 and 35.

03 Pass labels first and heights second to plt.bar.

WORKED EXAMPLE

Machine-mean bars

```
plt.bar(groups.index, groups.values)
plt.xlabel("Machine")
plt.ylabel("Mean target (kW)")
plt.title("Mean target by machine")
plt.show()
```

HOW TO READ IT

Each bar represents a group mean.

Use the same unit as target.

Expected bar heights: M1 = 15, M2 = 35.

WHAT TO DO

Functions and returned answers

01 A function is a named calculation. def starts its definition. Indented lines belong to it.

02 An argument supplies the input. return sends the result back to the caller.

03 In the test, keep every supplied function name and calculate from its input.

WORKED EXAMPLE

A practice function

```
def average_reading(values):
    readings = pd.Series(values)
    return float(readings.mean())

print(average_reading(raw["target"]))
print(average_reading(raw["load"]))
```

HOW TO READ IT

values receives the supplied column.

float returns a Python decimal number.

Expected: mean target 25.0; mean load 50.0%.

WHAT TO DO

Practice columns and test columns

01 Practice reading corresponds to test record_id. Practice machine corresponds to test equipment_id.

02 Practice load corresponds to test load_pct. Practice ambient corresponds to test ambient_c. target has the same name.

03 The calculations are the same. Use the column names in the file you are working with.

WHAT TO DO

Tables and lists of records

01 A DataFrame is a labelled table. A list of dictionaries stores the same rows as separate Python records.

02 table.to_dict("records") returns one dictionary per row. pd.DataFrame(records) turns those records back into a table.

03 The test supplies a conversion helper. You may use it when returning cleaned records.

WORKED EXAMPLE

The retained rows as records

```
practice_records = raw.to_dict("records")
print(practice_records[0])
restored = pd.DataFrame(practice_records)
print(len(restored))
```

HOW TO READ IT

[0] selects the first record.

The table has the same columns and values.

Expected: first record has reading 1 and target 10. Row count: 4.

WHAT TO DO

Missing values during conversion

01 The test helper uses astype(object) so cells can hold Python None.

02 notna() marks known values. where(mask, None) keeps those values and replaces missing cells with None.

03 The helper finishes with to_dict("records"). It is supplied so you do not have to invent a missing-value converter.

WORKED EXAMPLE

A helper for the original CSV

```
def as_records(table):
    objects = table.astype(object)
    return objects.where(table.notna(), None).to_dict("records")

original_records = as_records(source)
print(len(original_records))
```

source is the original uploaded CSV.

Expected: 7 original records, including the rows to remove.

WHAT TO DO

A cleaning function follows the same steps

01 def names the function and its input. Every indented line belongs to that function.

02 Build a table from the input, apply the earlier cleaning rules, and return records using the helper.

03 Calling the function with different records must calculate a new result.

WORKED EXAMPLE

The complete practice cleaning function

```
def revision_clean(records):
    table = pd.DataFrame(records)
    table = table.drop_duplicates("reading")
    table = table.dropna(subset=["target"])
    table = table.loc[table["load"].between(0, 100)].copy()
    middle = table["ambient"].median()
    table["ambient"] = table["ambient"].fillna(middle)
    return as_records(table)
```

The input is records, not a global CSV table.

Returns a new list of cleaned records. It does not print the answer.

WORKED EXAMPLE

Calling the cleaning function

```
cleaned_records = revision_clean(original_records)
print(len(original_records))
print(len(cleaned_records))
print(cleaned_records[1]["ambient"])
```

The original list remains available.

Expected: 7 original rows, 4 cleaned rows, replacement ambient 30.0.

WHAT TO DO

Summary values inside a dictionary

01 len(table) counts rows. target.min() and target.max() find the smallest and largest values.

02 A return dictionary gives each result a name: count, mean, minimum, maximum and group_means.

03 A grouped Series.to_dict() converts its equipment labels and means into another dictionary.

WORKED EXAMPLE

A summary function

```
def revision_summary(records):
    table = pd.DataFrame(records)
    values = table["target"]
    groups = table.groupby("machine")["target"].mean()
    return {"count": len(table),
            "mean": float(values.mean()),
            "minimum": float(values.min()),
            "maximum": float(values.max()),
            "group_means": groups.to_dict()}
```

Braces enclose key–value pairs.

Returns named numerical values and a dictionary of machine means.

WORKED EXAMPLE

Reading the returned summary

```
summary = revision_summary(cleaned_records)
print(summary["count"], summary["mean"])
print(summary["minimum"], summary["maximum"])
print(summary["group_means"])
```

Use a key to retrieve a named answer.

Expected: 4 and 25.0; 10.0 and 40.0; M1: 15.0, M2: 35.0.

WHAT TO DO

A Figure is an object you can return

01 plt.subplots() creates a Figure and its plotting Axes. fig holds the whole chart; ax holds the drawing area.

02 ax.scatter, ax.bar and ax.hist draw on those Axes. set_xlabel, set_ylabel and set_title add labels.

03 return fig gives the chart to the caller. plt.show() only displays it.

WORKED EXAMPLE

A function returning a scatter Figure

```
def revision_scatter(table):
    fig, ax = plt.subplots()
    ax.scatter(table["load"], table["target"])
    ax.set_xlabel("Load (%)")
    ax.set_ylabel("Target (kW)")
    ax.set_title("Practice load and target")
    return fig
```

fig and ax receive the two created objects.

The returned Figure contains four points from the practice CSV.

WHAT TO DO

Alphabetical order for group means

01 groupby("machine")["target"].mean() calculates one mean per machine.

02 sort_index() orders that Series by its labels. M1 comes before M2.

03 Use the same ordered Series for both the bar labels and heights.

WORKED EXAMPLE

A function returning ordered bars

```
def revision_bars(table):
    means = table.groupby("machine")["target"].mean()
    means = means.sort_index()
    fig, ax = plt.subplots()
    ax.bar(means.index, means.values)
    ax.set_xlabel("Machine")
    ax.set_ylabel("Mean target (kW)")
    ax.set_title("Practice machine means")
    return fig
```

sort_index sorts the group labels.

Expected bar order and height: M1 = 15, then M2 = 35.

WHAT TO DO

Ten histogram bins

01 The earlier two-bin example explained counting. The test asks for ten equal-width bins.

02 ax.hist(values, bins=10) divides the range from the minimum to the maximum into ten equal intervals.

03 For practice targets 10 to 40, each bin is 3 units wide. Empty bins still count as bins.

WORKED EXAMPLE

A function returning a ten-bin histogram

```
def revision_histogram(table):
    fig, ax = plt.subplots()
    ax.hist(table["target"], bins=10)
    ax.set_xlabel("Target (kW)")
    ax.set_ylabel("Number of observations")
    ax.set_title("Practice target distribution")
    return fig
```

bins=10 lets matplotlib calculate the edges.

Expected: 10 bars with total count 4, spanning targets 10 to 40.

WORKED EXAMPLE

Returning the three Figures in order

```
def revision_charts(records):
    table = pd.DataFrame(records)
    return [revision_scatter(table),
            revision_bars(table),
            revision_histogram(table)]

figures = revision_charts(cleaned_records)
plt.show()
```

HOW TO READ IT

Each helper returns one real Figure.

The list holds scatter, bar and histogram Figures in that order.

WHAT TO DO

The gap between group means

01 The group gap is the largest group mean minus the smallest group mean.

02 Our machine means are 15 and 35. The gap is 35 − 15 = 20.

03 This differs from the individual target range: 40 − 10 = 30.

WORKED EXAMPLE

Calculating the group gap

```
table = pd.DataFrame(cleaned_records)
means = table.groupby("machine")["target"].mean()
gap = float(means.max() - means.min())
print(gap)
```

max and min operate on group means here.

Expected: group gap 20.0 kW.

WHAT TO DO

A finding contains a value and a unit

01 A finding is a small dictionary, for example {"value": 25.0, "unit": "kW"}.

02 The outer dictionary names the finding. The inner dictionary stores its number and unit.

03 Use observations for count. Use the target unit for mean, minimum, maximum and group_gap.

WORKED EXAMPLE

A helper for a value with its unit

```
def with_unit(value, unit):
    return {"value": value, "unit": unit}

example = with_unit(25.0, "kW")
print(example["value"], example["unit"])
```

Expected: 25.0 kW.

HOW TO READ IT

The function returns one inner dictionary.

WORKED EXAMPLE

Returning the five practice findings

```
def revision_findings(records):
    result = revision_summary(records)
    means = pd.Series(result["group_means"])
    gap = float(means.max() - means.min())
    unit = "kW"
    return {"count": with_unit(result["count"], "observations"),
            "mean": with_unit(result["mean"], unit),
            "minimum": with_unit(result["minimum"], unit),
            "maximum": with_unit(result["maximum"], unit),
            "group_gap": with_unit(gap, unit)}
```

The count has a different unit.

Returns five findings, each containing value and unit.

WORKED EXAMPLE

Checking the returned findings

```
findings = revision_findings(cleaned_records)
print(findings["count"])
print(findings["mean"])
print(findings["group_gap"])
```

Two keys reach a nested value.

Expected values: count 4, mean 25.0, group gap 20.0.

WHAT TO DO

Practice: another input to your functions

01 Use original_records[:2] to select the first two original records. Clean this smaller input.

02 Call your summary, charts and findings functions on that result.

03 Predict the retained count, mean and group gap before running.

Practice answer

01 The two input records have unique IDs, known targets and valid loads. Both remain.

02 Mean target: 15. Only M1 remains, so group gap is 0. The missing ambient value becomes 20.

03 The functions calculate from the supplied argument, even when its length changes.

WHAT TO DO

What a filled value can and cannot tell us

01 A median replacement is an estimate, not a recovered measurement. It can change summaries and patterns.

02 Keep the original data and state the replacement rule.

03 A pattern in synthetic practice data does not establish real equipment performance.

WHAT TO DO

An identifier is not a clock

01 reading and record_id identify observations. Their numbers do not tell us elapsed time.

02 A time trend needs an actual timestamp or a documented time sequence.

03 An equipment mean describes the observed sample. Investigate representative measured data before making operational decisions.

WHAT TO DO

Test 1 revision checklist

01 Read and clean the CSV, then return records without changing the input.

02 Return a summary dictionary, three Figure objects and five findings with units. Check the practice outputs.

03 Explain the limits of imputation, the difference between association and cause, and why an identifier is not time.

WHAT TO DO

Recording interpretation answers

01 Read each question and its labelled answer options. Choose the option supported by the data or code.

02 In DECISIONS, keep the question identifier unchanged and replace its blank value with your chosen option letter.

03 These letters identify answer choices. The portal assigns your test files automatically.

WORKED EXAMPLE

A dictionary entry for a practice question

```
DECISIONS = {"practice": "B"}
print(DECISIONS["practice"])
```

HOW TO READ IT

"practice" is a question identifier.

"B" is that question’s selected option.

Syntax example only: B represents a chosen option, not a test version.

WHAT TO DO

Test-day instructions

01 The practice section ends here. Keep your revision notebook for study.

02 The following slides explain the timed test files, individual work and submission.

03 Use the portal-issued notebook and CSV for the actual test.

TEST ACCESS

Your test files on test day

01 Downloads unlock at the scheduled start time. Open the link or scan its QR code.

02 Enter your name, Google email and Student Registration Number.

03 Download your notebook and engineering_data.csv.

TEST FILES

Open your test files

Direct URL (click or copy into your laptop browser):

https://iuea-test-entry.jothammartin-wambi.workers.dev/?test=CML3112%7CTEST1

WHAT TO DO

Opening the notebook and CSV

01 In Colab: File → Open notebook → Upload. Select the downloaded notebook.

02 Save a copy in Drive. Connect the session.

03 In Files on the left, upload engineering_data.csv.

WHAT TO DO

One notebook contains all answers

01 Code tasks have a prompt, function name and marks.

02 Six interpretation questions: record your chosen option letters in DECISIONS.

03 Submit one notebook snapshot using the final button.

WHAT TO DO

How the 100 marks are allocated

01 Cleaning 20. Summary statistics 15.

02 Three labelled charts 15. Five numerical findings 20.

03 Structured interpretation 30.

WHAT TO DO

Order of work

01 Load the raw CSV. Keep the original data.

02 Clean records before calculating summaries, charts and findings.

03 Answer the six interpretation questions.

WHAT TO DO

The three-hour session

01 First 15 minutes: setup and file checks.

02 Next 120 minutes: individual coding tasks.

03 Next 25 minutes: interpretation and checking.

04 Final 20 minutes: prepare the snapshot and submit the form.

WHAT TO DO

Before submitting

01 Restart the Colab session and run cells from the top.

02 Upload the same CSV again if the new session needs it.

03 Fix errors and run the final submission cell.

WHAT TO DO

Submission and confirmation

01 Click Submit test at the end of your notebook.

02 Open the generated form link using the same Google account. Check your details.

03 Press Submit before the deadline. Keep the confirmation.

WHAT TO DO

How grading works

01 The lecturer’s grader runs your submitted code on unseen data.

02 The private workbook records identity, submission time, status and marks.

03 A technical or identity problem remains pending until resolved.